A Tree-based Approach to Analyse the Relationship between Avian Taxonomic Distance and Vocalization Similarities

April 23, 2026

Introduction and Project Goal

We were motivated to see the correlation between closeness of avian classification, and the similarity in their songs and calls.

This is to compare the sounds that birds make and how similar they are, to how close the birds are genetically related. By cross-analyzing these two sets of data, we want to answer the hypothesis if or if not the way they sound can be determined purely from the taxonomic distance of birds.

Thus, it raises our research question: **how close do bird species relate to each other based on the similarity of their vocalization?**

Used datasets

The data we obtained are from the Xeno-Canto API. Since analyzing vocalizations of all avians is infeasible, we focused on some members of the three families paridae, fringillidae, and phylloscopidae under the order Passeriformes, as these are some families of the most numbers of species in Passeriformes and so are more representative; we also used recordings from some species of the orders Strigiformes and Piciformes, in order to increase the distances between species. Since recording files are generally large, we had to filter for only 10-30 second recordings, and set both the maximum species in each family and the maximum number of recordings for each species to 10. To improve our analysis quality, we filtered for only recordings of `A` quality and type `song`, since this is the most iconic of each bird and including multiple types of vocalization can introduce much unwanted noise. Due to these constraints, some species had less than 10 recordings, and the fringillidae family containing only 8 species; but we considered it necessary because even though reducing quality requirements would increase the number of recordings, it would make analysis much harder.

An example of the returned recording object by using the API has the format shown below. However, for this project, in building our `taxonomy_tree`, we only used `fam`, `gen`, `sp`, and `en`.


```
{
                "id": "694038",
                "gen": "Troglodytes",
                "sp": "troglodytes",
                "ssp": "",
                "grp": "birds",
                "en": "Eurasian Wren",
                "rec": "Jacobo Ramil MIllarengo",
                "cnt": "Spain",
                "loc": "Sisalde, Ames, A Coru\u00f1a, Galicia",
                "alt": "30",
                "type": "song",
                "sex": "male",
                "stage": "adult",
                "method": "field recording",
                "url": "\/\/xeno-canto.org\/694038",
```

```
            "file": "\/\/xeno-canto.org\/694038\/download",
            ... (unused information ommitted) ...
        }
```

Note that the Xeno-Canto API also provides information on the avian subspecies, the `ssp`. We will ignore this classification and compute all the subspecies' vocalizations as the same species. This is because often the classification of subspecies is debatable, to the extent that some may be classified as a distinct species later on. In addition, the number of recordings for different subspecies are unbalanced, which may make our data analysis less fair.

We used `download_recordings.py` for downloading the recordings. The downloaded data are stored in the folder `bird_data`, with individual recordings stored in the corresponding folders of each family. Information of recordings are stored in `bird_metadata.csv`, where each row is in the format of `family, genus, species, latin_name, common_name, xc_id, local_file_path, quality, duration` . The taxonomic information of individual bird species is stored in `bird_taxonomy.csv`, with the format `family, genus, species, latin_name, common_name`. To process the recordings, we defined the `RecordingData` class in `classes.py` using Librosa's sound analysis. The distance in taxonoy tree is calculated using `get_distance_between` method (details described later on). In `process_recordings.py`, we extracted all sound features and distances and the processed data is stored in `comparison_data.csv`. This is the data used for graphing and analysing later on.

To produce our final visualization, wikimedia API is also used to obtain avatars of required species in the tree.

Computational Overview

The Taxonomy Tree

The Taxonomic structure of avian classification resembles a tree data type, where the internal nodes of the tree represent the classification ranks. To simplify the tree, since all the birds of the same class, we ignored the Domain, Kingdom, and Phylum, so that the root of our tree is class Aves, with its children being the order, whose children are the family, then genus, and the leaves being the species.

All the trees' nodes contain its Latin name `_root`, its rank `_rank`, and its subtrees `_subtrees`. Since we want to calculate the distance between species, we included the `_parent` for each node to make it a doubly-linked tree, which is automatically set upon initializing, so that the tree is traversable from both directions.

They also contain an optional instance attribute `_species` of type `Species`, which is only needed for the leaves and represent the Species and its vocalization detail. This is represented by the `RecordingData` class that includes the paths of all raw recordings and the processed vocalization data of each species.

`TaxonomyTree`'s method `get_distance_between` calculates the distance between different species using the path from the species, which is the leaves of the tree to the room. For example, firstly, consider a part of the tree:

```
Aves (class)
    Charadriiformes (order)
        Scolopacidae (family)
            Calidris (genus)
                Calidris pygmaea (species)
            Scolopax (genus)
                Scolopax minor (species)
```

To calculate the taxonomic distance between Calidris pygmaea and Scolopax rusticola, the method first finds their common "ancestory", where they first share a common ancestor of the family Scolopacidae, so their distance should be defined as 2. The further down the branching point, the closer the kinship is for the species and the smaller the "taxonomic distance".

`process_recordings.py` builds the tree and analyses the recording data. To build the tree, each row of `bird_taxonomy.csv` is used as an input to `add_species` of the `RecordingData` class. The method first checks for the correct order of the species based on its family, then uses the helper method `_get_or_create_child` to dynamically create or find the parent node. This helper checks if the given node exists, and creates a new node only if it doesn't. It finally creates the new species node and sets the `RecordingData` object.

Vocalization similarity

To analyse the recording similarity, the `RecordingData` class is defined so that all features of files at the input file path are extracted upon initialization. The private `_extract_feature` method calculates the features of each recording using the Librosa Library, which is commonly used for vocal analysis. For pre-processing the recordings, the method first loads in the file with `librosa.load` and removes leading and trailing silence with `librosa.effects.trim` to narrow down the range required to analyse. `librosa.effects.preemphasis(y, coef=0.97)` applies filter to emphasize on higher frequency and reduces lower frequency noise, which increases the finess of comparisons. It then normalizes the loudness between different recordings to $[-1, 1]$. To extract the features, `librosa.feature.mfcc(y=y, sr=sr, n_mfcc=13, fmin=100)` is used to calculate the first 13 Mel-frequency Cepstral Coefficients, which represents the recording timbre. `librosa.pyin` estimates the pitch; `librosa.feature.spectral_centroid()` and `librosa.feature.spectral_bandwidth()` calculate the frequency and brightness, and `librosa.feature.rms(y=y)` estimates the overall loudness. The average of all features are then taken using `average_features` and stored as a dictionary that is more easily used afterwards.

The similarity of calls between two species is calculated using cosine similarity or Euclidean distance between their feature vectors (Pedregosa et al., 2011).

The `get_feature_vector` method in `RecordingData` converts the averaged feature dictionary into a one-dimensional vector in the order: MFCC coefficients (8 values), pitch mean, spectral centroid mean, spectral bandwidth mean, and RMS mean, resulting in a 12-dimensional feature vector for each species.

Before computing similarity, Z-score normalization is applied to all feature vectors using `normalize_features` in `sound_analysis.py`. For each feature dimension i, the mean μ_i and standard deviation σ_i are computed across all species, and each value is standardized as:

$$z_i = \frac{x_i - \mu_i}{\sigma_i}$$

This ensures that features with large numeric ranges (e.g. pitch in Hz) do not dominate over features with smaller ranges (e.g. MFCC coefficients).

The cosine similarity between two normalized feature vectors $\mathbf{a}$ and $\mathbf{b}$ is defined as:

$$\text{cosine_similarity}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \, \|\mathbf{b}\|} = \frac{\sum_{i=1}^{n} a_i b_i}{\sqrt{\sum_{i=1}^{n} a_i^2} \cdot \sqrt{\sum_{i=1}^{n} b_i^2}}$$

This returns a value between -1 and 1, where values closer to 1 indicate more similar vocalizations.

The Euclidean distance (SciPy, 2024) between two feature vectors is defined as:

$$d(\mathbf{a}, \mathbf{b}) = \sqrt{\sum_{i=1}^{n} (a_i - b_i)^2}$$

Smaller distances indicate more similar vocalizations. Both metrics are implemented in `sound_analysis.py` and used in `process_recordings.py` to compute pairwise comparisons across all species, with the results stored in `comparison_data.csv`.

Interactive Visualizations

To directly visualize the similarities, a tree visualizer representing the taxonomy tree of the selected species is drawn, so that when the avatar of a species is clicked, the other species' colors update to take a color in the blue to red range, where the more red represents higher similarity and blue represents lower similarity.

The setup of the tree is split into a few parts:
visualize_tree.py - main handler
import_images.py - image fetcher through Wikimedia API
image_fallback.py - image caching and backup
tree_style.py - style for UI elements

To generate a tree visualizer, we first create a TaxonomyTree object and calculate its comparison data using the computation process explained above. The computed comparison data will take on this format:

```
example_comparison = {
    'item1': str,
    'item2': str,
    'similarity': float,
    'distance': int,
}
```

where `similarity` equals to the vocalization similarity between two birds while `distance` represents the taxonomic distance between them. It is also recommended to run `image_fallback.load_url` to cache bird thumbnails in a comprehensive dict from a preset CSV file for faster loading.

Based on the above, calling the main wrapper `visualize_tree.run` will take the TaxonomyTree as its first argument, and a compiled **list** of all comparisons between every bird in the TaxonomyTree as its second argument.

From there, the tree generation is done in the following steps:

1. **visualize_tree.calculate_tree_layout** loops through the tree object recursively top to bottom from its root and identifies the edges and leaves of the tree. The position of the tree is (x,y) based on the recursion depth and number of leaves.

For leaf nodes, x is simply the current number of leaves found (0, 1, 2, ...), and y is the depth PLUS `round(len(leaves) % 2)`, so the leaves are spaced vertically for cleaner organization.

For parent nodes, x is `sum(children_x_coords) / len(children_x_coords)` so it's always placed in the middle, and y is just the depth starting from 0.

Thus, a tree with 1 root and 3 leaves will have the coordinates: (1, 0) for root and (0, 1), (1, 2), (2, 1) for the leaves respectively.

Additionally, during each recursion at a leaf node, its bird thumbnail is either retrieved from **image_fallback.py** if there exists a preset, or fetched with **import_images.py** from a Wikipedia page with the Wikimedia API.

2. Once we found the leaves, edges, and positions, **visualize_tree.get_similarity_color_rgba** and **visualize_tree.build_similarity_map** handles the color computation from similarity. **visualize_tree.build_similarity_map** builds a dict mapping of each comparison between 2 birds and calculates for the max, min and min-max difference of all similarity scores. **visualize_tree.get_similarity_color_rgba** then finds the ratio (0-1) of a particular similarity divided by the min-max difference. The ratio is mapped on a blue to red RGB gradient, where 0 is blue and 1 is red.

3. We use **plotly**'s **dash** and **DashIconify** modules to create the tree visualizer in the browser. The style of the UI elements is configured by **tree_style.STYLESHEET**.

4. Finally, **visualize_tree.on_input** handles user interaction, where clicking on a leaf node that represents a bird species will update the color mapping and play a random sound with **visualize_tree.play_random_mp3**. The sound is randomly selected from downloaded recordings whose size is at most 1MB to avoid out-of-memory error. At the end, we also call **visualize_tree.save_img_url** to update the preset CSV file to store new bird thumbnails for future use.

For analysing data, three graphs of the analysed data are drawn, all of which has their horizontal axis representing the taxonomic distance and the vertical axis representing the vocalization similarity:
1. The main graph displays all pairs of species comparison data,
2. The "zoomed-in" graph that focuses on a single species' comparison data with all the others,
3. A bar and whisker plot showing the standard deviation of comparison data.

For 1) and 2), the graph is either drawn as a static or interactive scatter plot with **visualization.py**. We use **plotly** for the interactive graph and **matplotlib** for the static graph. Both use a list of comparison data dictionaries between two birds formatted in the same way as what the tree visualizer required above.

For third graph is a static plot that shows the mean of the similarities for each taxonomy distance.

—

Libraries and API used for visualization:

1. Dash and Dash Cytoscape

Dash is the web framework used to build the tree visualizer interface. Dash Cytoscape is a component library responsible for creating the UI networks (HTML/CSS) and handling user input that make the visualizer interactive without needing javascript.

2. Plotly

Plotly is the core interactive graphing library used to generate the visualizers with user input.

3. Matplotlib

Matplotlib is alternative to Plotly used to create static graphs that don't require user interaction.

4. Wikimedia API

It is Wikipedia's official api for web requests for Wikipedia pages. We used this API to fetch thumbnail image URLs from Wikipedia pages about different bird species. However, the rate is limited, so caching locally was a faster alternative.

Instructions for obtaining datasets and running program

The raw recording data (stored in `bird_data`) consists of hundreds of files, which is infeasible to be quickly ran and obtain the processed results. Instead, download the file `partial_bird_data.zip` and unzip into the same path as `main.py`. Alternatively, go to PyCharm and uncomment out the first two lines that handles downloading sample audio. `main.py` can be directly run in the Python console. An analysis of the data is printed in the console, which reads that

```
=====================================================================
Taxonomic Distance vs Vocalization Similarity correlation analysis
=================================================================
Pearson r   : -0.2441
R^2         : 0.0596  (5.96%)
p-value     : 0.000000
=====================================================================
```

Seen below, figues 1 and 2 will be displayed in the web browser, which shows scatter plot of data between single species with all others, and the full chart. Figure 3 is the interactive chart that also displays in the browser after clicking on the link of the server form the console. You can zoom in and out of the tree, and select certain species. Doing so will play a recording of that species, and also color all other species, where red represents a more similar vocalization, and blue represents less similar. Figure 4 shows the interactive coloring. After stopping the taxonomy tree in the console by ctrl+C, a standard deviation graph is shown, like that of figure 5.

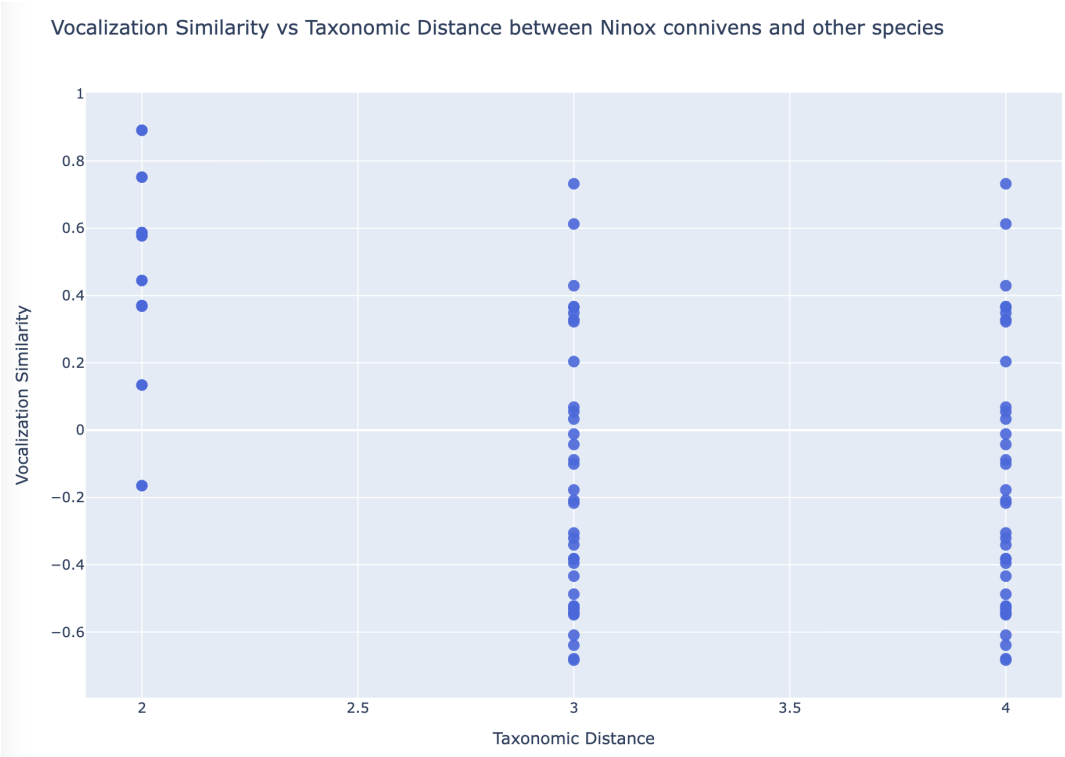

Figure 1: Comparison data of single species with all others

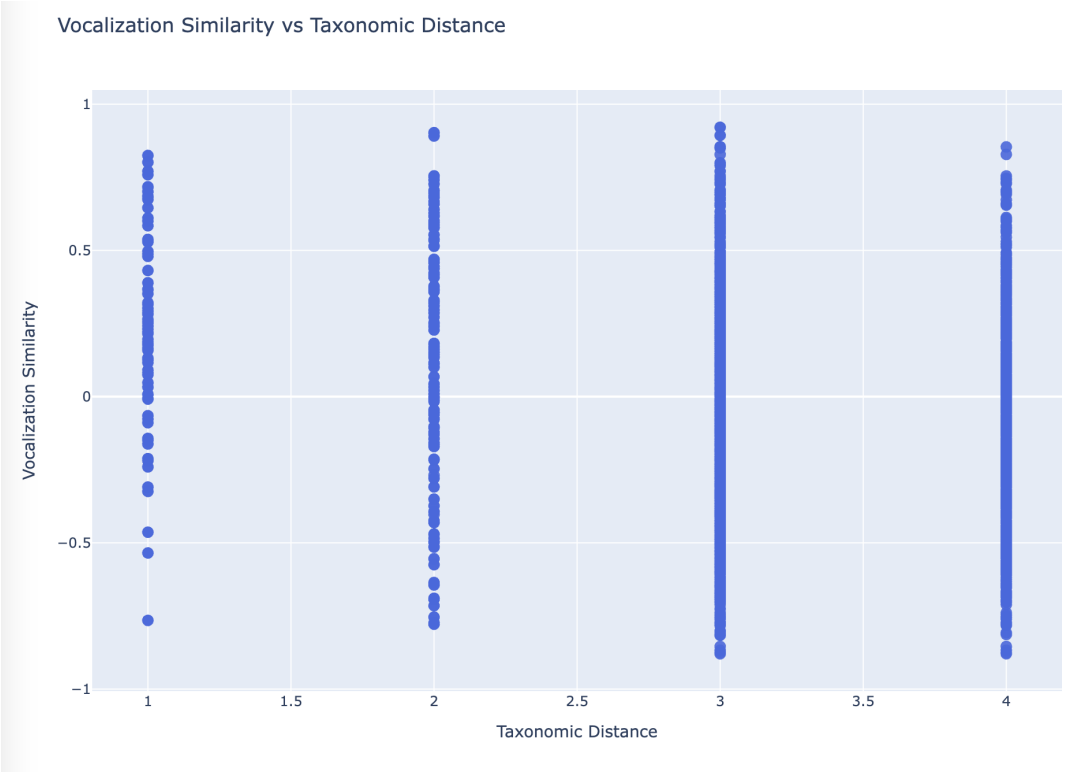

Figure 2: Comparison data of all species

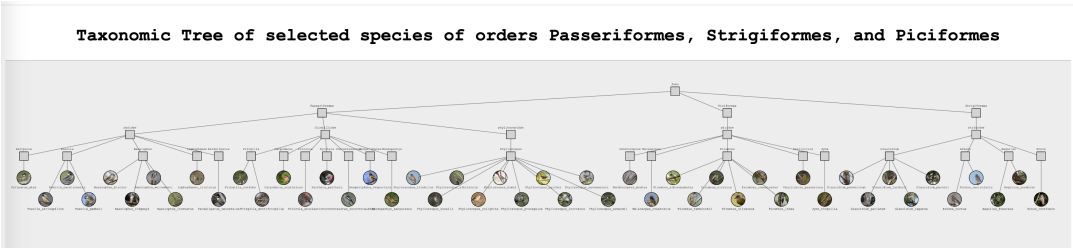

Figure 3: Interactive chart

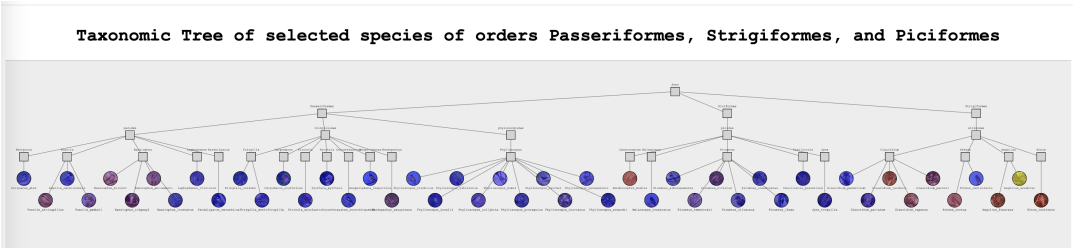

Figure 4: Interactive chart - similarity coloring after pressing on the species

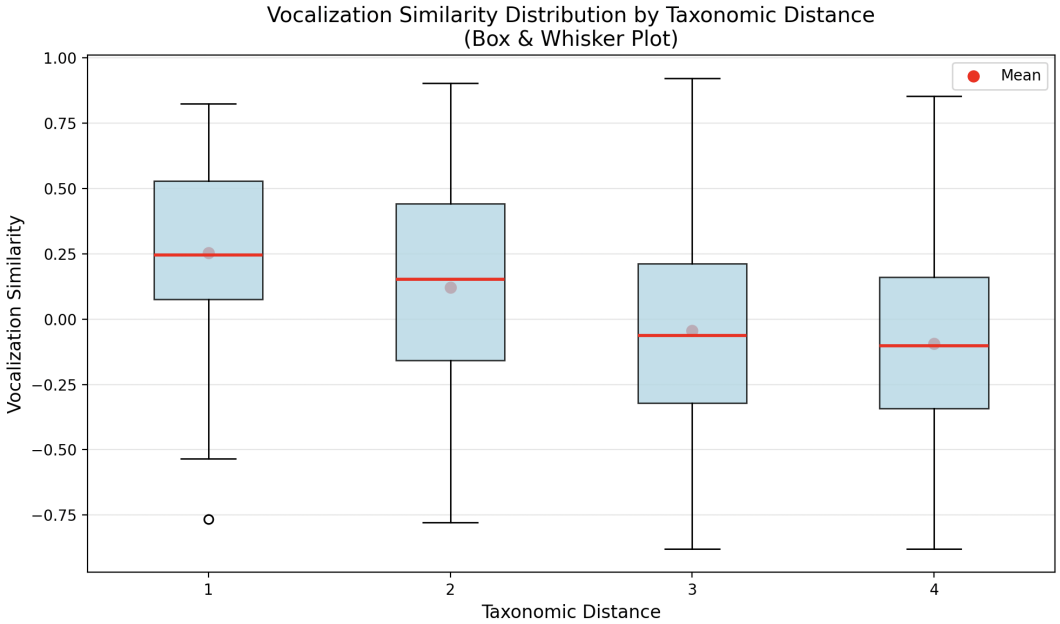

Figure 5: mean and standard deviation of data

Changes to project plan

- We removed the duration of recording from the part of the recording analysis, since it's dependent on the recorder but not the species. This reduces the noise of the data so the relationships are clearer.
- We only included at most 10 species in each family to have a balance between a less biased average and feasible amount of data for processing. Similar to mentioned before, some recordings were filtered out since they were more than 10 MB and is infeasible.
- In our original plan, we only included the Passeriformes order; we now also included the orders Strigiformes and Piciformes. This is because originally since all species are from the same order, the maximum distance on the taxonomy tree is 3. We wanted to have a more significant comparison, so that the new orders now provided a

maximum distance of 4. This increased the range of the independent variable, making our analysis results more convincing.

Discussion

In our project, we used avian species from selected orders to create their taxonomy tree, using which we were able to calculate the taxonomic distance. Using their song vocalization recordings from Xeno-Canto, we used Librosa to extract their features. We then calculated the similarity between each species vocalization, and visualized their data.

Based on our processed data, a total of 2259 comparisons were performed. Using the complete processed data, the Pearson correlation coefficient is calculated to be -0.2441, while the coefficient of determination R^2 is 0.0596. The p-value is less than 0.001.

This means there is a negative correlation between Taxonomic Distance and Vocalization Similarity, and this relationship is statistically highly significant ($p < 0.001$). Also, 5.96% of the variation in the vocalization similarity is determined by changes in the taxonomic distance, which is able to directly answer our research question.

This fits our expectations that there may be a correlation between the two variables, but it might not be a causal relationship. Since taxonomic classifications are based on and reflects common features, species of lower distances may have more similar habitats, Syrinxes (organ used for vibration to vocalize), and behaviors, so their vocalization would be more similar.

While this may be less apparent in the scatter plots, where the similarities have a large range for all the taxonomy distances, the correlation can be quite directly observed in the standard deviation graph. The mean similarity decreases and spread of similarity increases as distance increases, with the highest mean at when distance is 1. In addition, the visualization also clearly shows that species that are closer on the tree is colored more red.

The main limitation is that vocalization records are not highly standardized; for example, one recording may have more than 10 minutes of different types of calls with high noise in the background, another only 5 seconds of high quality recording.

This makes processing high quality output difficult, which leads to our potential area of future exploration. It is the extraction and analysis of recordings using wavelets to detect the sounds (Drange, 2020), which can significantly reduce noise and improve collected data quality. As a result, we expect a higher correlation between data in this approach. Since it includes LLMs so is not less easily approachable than Librosa, and it requires much larger scale computations that we didn't have access to, we weren't able to implement it into our code.

In addition, our current code only analysed the similarities between song vocalizations. To make the result more rigorous, other vocalization types such as calls and of different individuals like male, female, or immature can be compared. The Euclidean distance of each can be taken and used to calculate the comparison data.

References

"Application Programming Interface (API v3)". Xeno-Canto, xeno-canto.org/explore/api. Accessed 4 Mar. 2026.

"Dash Cytoscape — Dash for Python Documentation — Plotly." Plotly.com, 2026, dash.plotly.com/cytoscape. Accessed 2 Apr. 2026.

Drange, Geir. "Extracting Bird Song Signatures with Wavelets." Kaggle, Kaggle, 2 Aug. 2020, www.kaggle.com/code/mistag/e bird-song-signatures-with-wavelets/notebook.

"Librosa." Librosa, librosa.org/doc/latest/index.html. Accessed 4 Mar. 2026.

Matplotlib. "Users Guide — Matplotlib 3.5.0 Documentation." Matplotlib.org, matplotlib.org/stable/users/index.html.

"ML169755 Blue Jay." Macaulay Library, macaulaylibrary.org/asset/169755. Accessed 3 Mar. 2026.

Pedregosa, F., et al. "Scikit-learn: Machine Learning in Python." Journal of Machine Learning Research, vol. 12, 2011, pp. 2825–2830, scikit-learn.org/stable/modules/metrics.htmlcosine-similarity. Accessed 2 Apr. 2026.

"scipy.spatial.distance." SciPy, docs.scipy.org/doc/scipy/reference/spatial.distance.html. Accessed 2 Apr. 2026.

"Wikimedia APIs - MediaWiki." MediaWiki, 2022, www.mediawiki.org/wiki/Wikimedia_APIsDocumentation. Accessed 2 Apr. 2026.